your app (unless you specifically disable them). For example, take this simple Razor template that posts back to the same Razor Page:

```
<form method="post">
    <label>Amount</label>
    <input type="number" name="amount" />
    <button type="submit">Withdraw funds</button>
</form>
```

When rendered to HTML, the antiforgery token is stored in the hidden field and is posted back with a legitimate request:

```
<form method="post">
    <label>Amount</label>
    <input type="number" name="amount" />
    <button type="submit" >Withdraw funds</button>
    <input name="__RequestVerificationToken" type="hidden"
value="CfDJ8Daz26qb0hBGsw7QCK"/>
</form>
```

ASP.NET Core automatically adds the antiforgery tokens to every form, and Razor Pages automatically validates them. The framework ensures that the antiforgery tokens exist in both the cookie and the form data, ensures that they match, and rejects any requests where they don’t.

If you’re using Model-View-Controller (MVC) controllers with views instead of Razor Pages, ASP.NET Core still adds the antiforgery tokens to every form. Unfortunately, it *doesn’t* validate them for you. Instead, you must decorate your controllers and actions with the `[ValidateAntiForgeryToken]` attribute. This ensures that the antiforgery tokens exist in both the cookie and the form data, checks that they match, and rejects any requests in which they don’t.